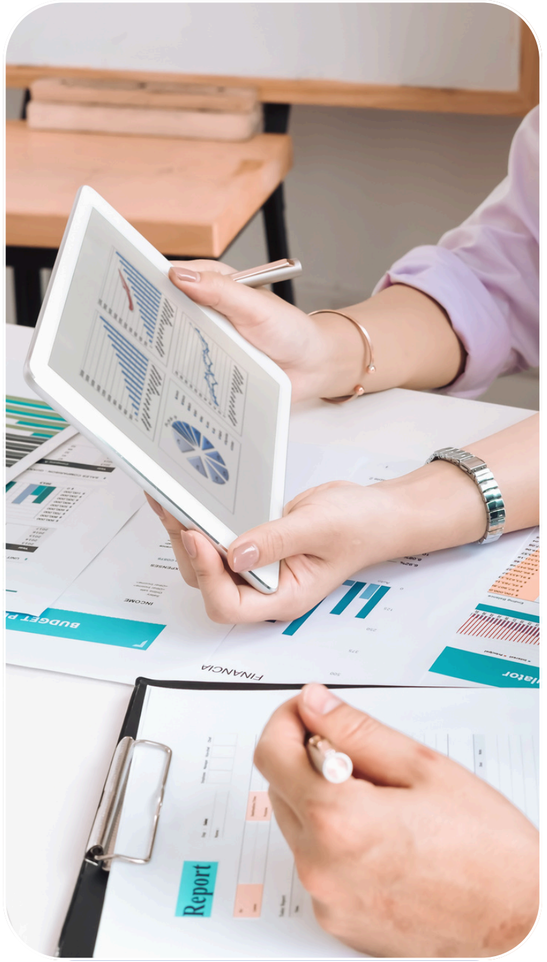
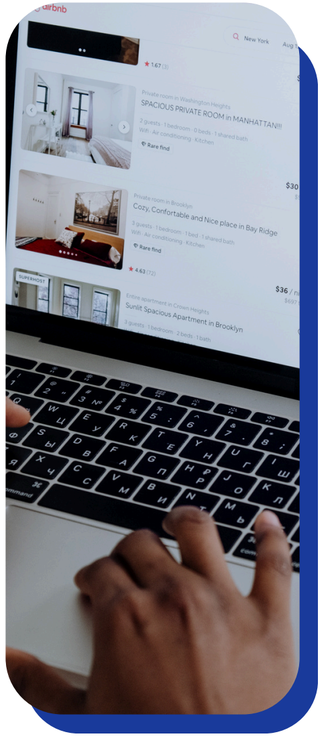
Smart Campus

Plateforme numérique de gestion de la vie universitaire





4DAS_I11

Présenté par:

Chadha Dalhoumi
Bechir Haboubi
Yassine Gharbi
Dhia Trabelsi

2026-2027

Problématique

Comment rendre les services du campus plus accessibles, coordonnés et faciles à suivre ?



01 Situation actuelle

- Demandes de salles par e-mail
- Événements annoncés sur des pages dispersées
- Annonces noyées dans les discussions
- Objets perdus signalés sur papier ou dans des groupes

02 Difficultés observées

- Conflits de réservation
- Demandes sans suivi clair
- Étudiants mal informés
- Gestion des places limitée
- Objets difficiles à restituer

03 Question centrale

Comment centraliser ces démarches dans une seule plateforme, tout en assurant le suivi, la validation et la coordination entre services ?

Notre solution

Une plateforme unique pour gérer les ressources, les clubs, les événements, les annonces, les objets perdus et les notifications

Réservations

- rechercher une salle et du matériel ;
- vérifier la disponibilité
- créer une demande ;
- approuver, refuser ou annuler.

SMART CAMPUS

Événements

Clubs + inscriptions

- gérer les clubs et les adhésions ;
- créer un événement ;
- gérer les inscriptions ;
- gérer la liste d’attente.

Annonces

créer et publier des annonces ;

Notifications

Messages + rappels
recevoir des notifications et des rappels.

Objets perdus

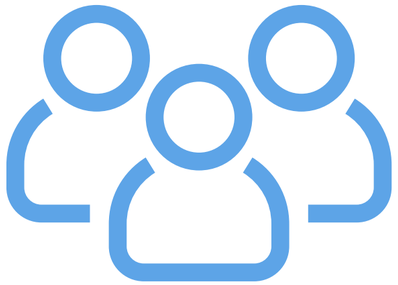
Déclaration + retour

Acteurs du système :



Étudiant

- Consulter, s’inscrire, rejoindre un club, déclarer ou réclamer un objet



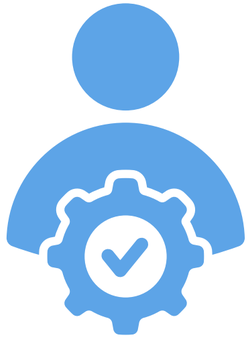
Personnel STAFF

- Créer certaines réservations, activités et annonces



Administrateur

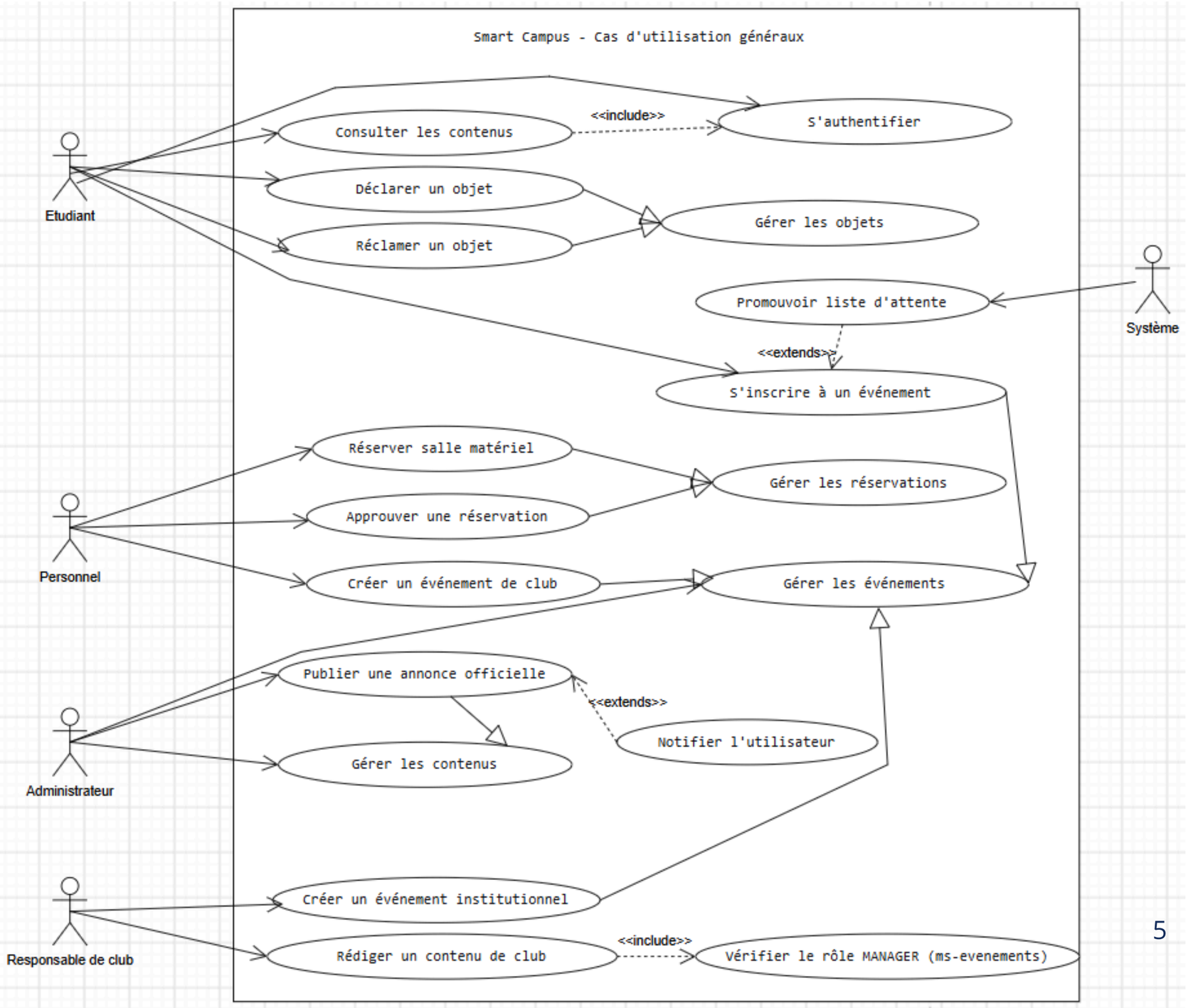
- Approuver, valider et superviser



Responsable de club

- Gérer son club, ses événements et ses contenus

Diagramme de cas d’utilisation global



Découpage en microservices

Microservice	Responsabilité	Base	Justificationde BD
ms-reservations	Salles, matériel et réservations	PostgreSQL	Transactions et prévention des conflits temporels
ms-evenements	Clubs, événements et inscriptions	MySQL	Relations, inscriptions et contrôle de capacité
ms-annonces	Annonces et articles	H2	Base légère pour le prototype
ms-objets-perdus	Objets, réclamations et restitutions	MongoDB	Attributs variables selon le type d'objet
ms-notifications	Notifications, préférences et rappels	H2	Notifications, préférences et rappels du prototype

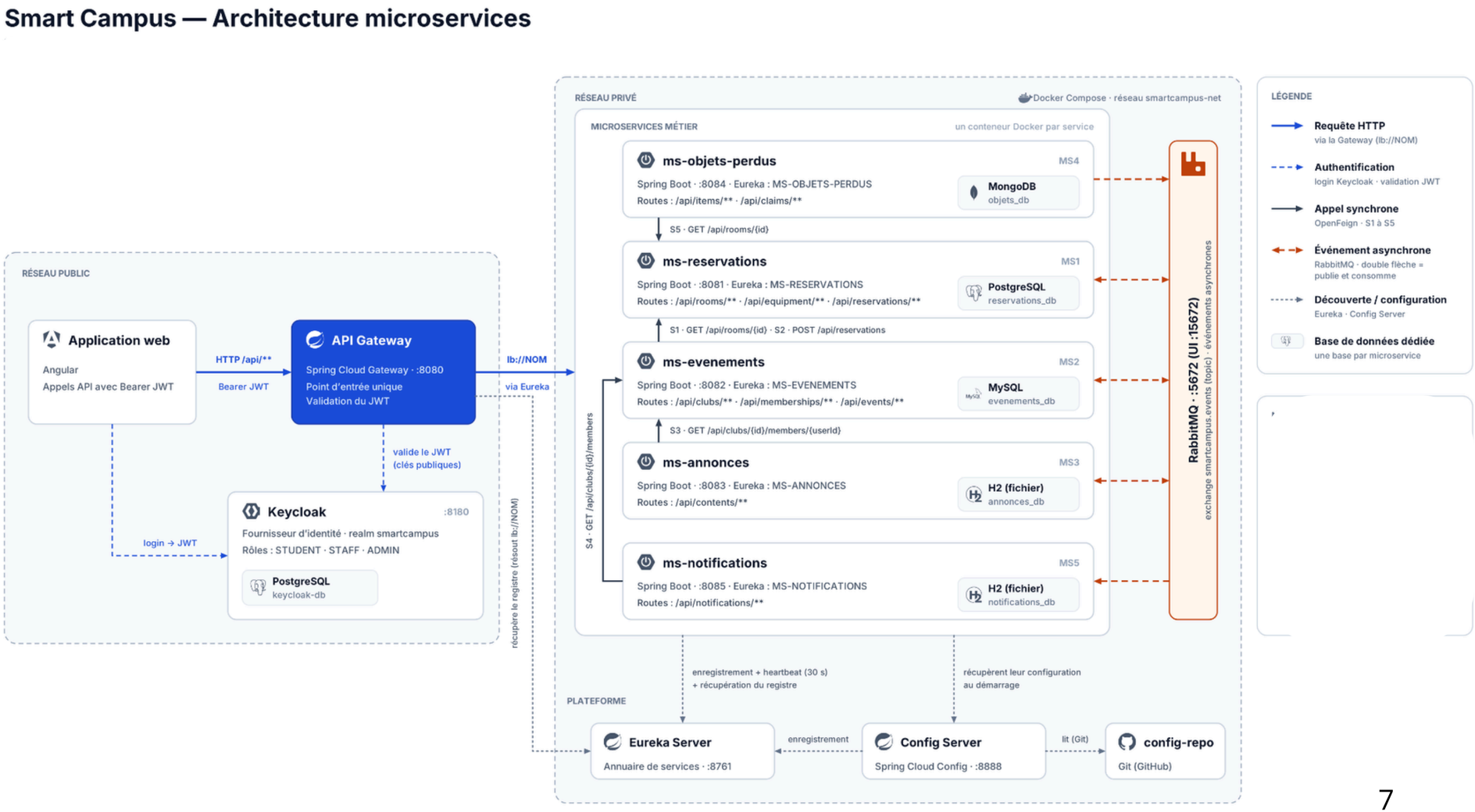
Principe appliqué

- une base indépendante par service
- aucun accès direct à la base d'un autre service
- échanges uniquement par API ou événements.

Architecture globale

Technologies principales

- Frontend : Angular
- Backend : Spring Boot
- API Gateway : Spring Cloud Gateway
- Découverte : Eureka Server
- Configuration : Config Server
- Sécurité : Keycloak
- Messagerie : RabbitMQ
- Conteneurisation : Docker Compose



Microservice Réservations

Une gestion centralisée des salles, du matériel et des demandes de réservation.

CE QUE LE SERVICE PREND EN CHARGE

Des ressources disponibles, sans conflits de réservation.

Le service vérifie les créneaux, la capacité des salles et la quantité de matériel avant de créer une réservation.

SALLES

Consulter les salles, leur capacité et leur disponibilité.

MATÉRIEL

Consulter les équipements et les quantités disponibles.

RÉSERVATIONS

Créer une demande avec une salle et le matériel nécessaire.

VALIDATION

Approuver, refuser, annuler ou faire expirer une réservation.

Base de données : PostgreSQL • Communication avec MS Événements lors de la création d’un événement

Acteurs et API

Les routes REST exposent les principales actions du microservice.

LES ACTEURS

ÉTUDIANT Consulte les salles et suit les réservations liées à ses activités autorisées.

STAFF Consulte les ressources et crée une demande de réservation.

ADMIN Gère les ressources et approuve, refuse ou annule les réservations.

EXEMPLES D'ENDPOINTS

RESSOURCES

GET	/api/rooms	Consulter les salles
GET	/api/equipment-types	Consulter le matériel
GET	/api/availability	Vérifier la disponibilité

RÉSERVATIONS

POST	/api/reservations	Créer une réservation
GET	/api/reservations/{id}	Consulter une réservation
POST	/api/reservations/{id}/approve	Approuver une réservation
POST	/api/reservations/{id}/reject	Refuser une réservation
DELETE	/api/reservations/{id}	Annuler une réservation

GET = consulter • POST = créer ou exécuter une action • DELETE = supprimer/annuler

Comment le microservice fonctionne

Les diagrammes résument les données principales et le scénario de création d'un événement.

DIAGRAMME DE CLASSES

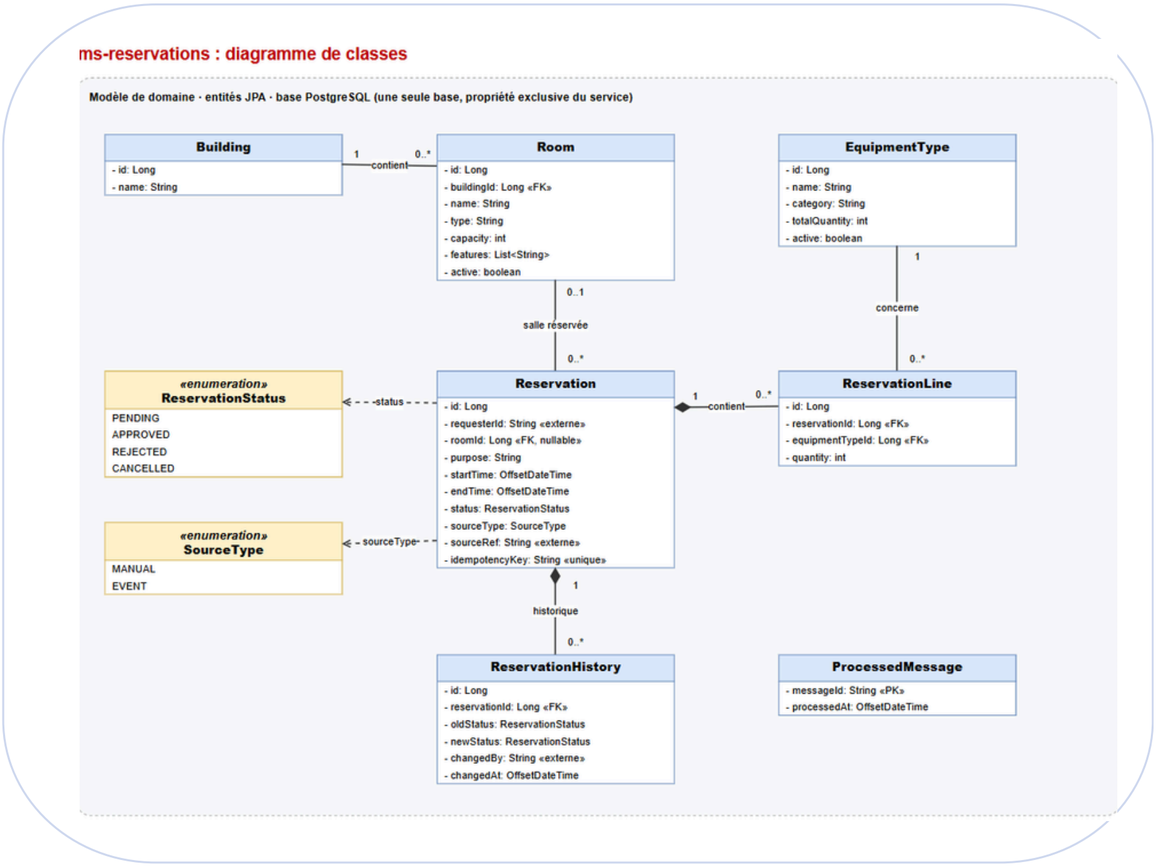
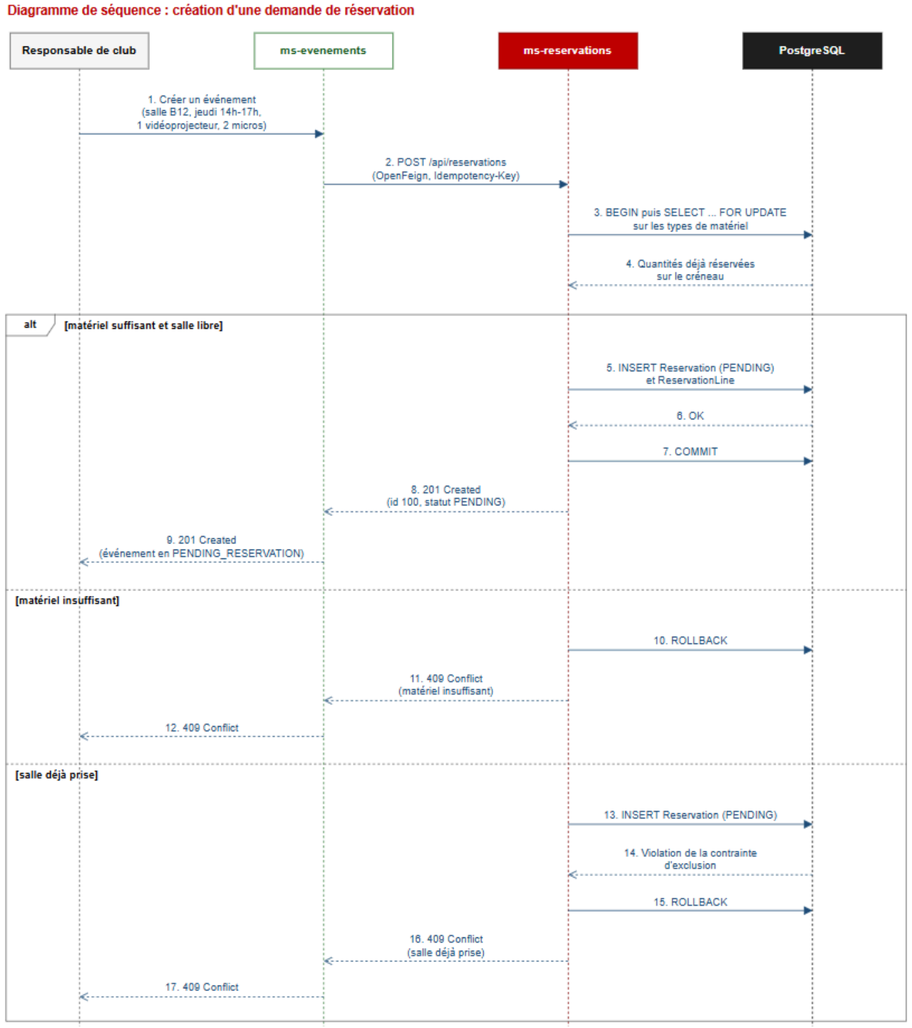
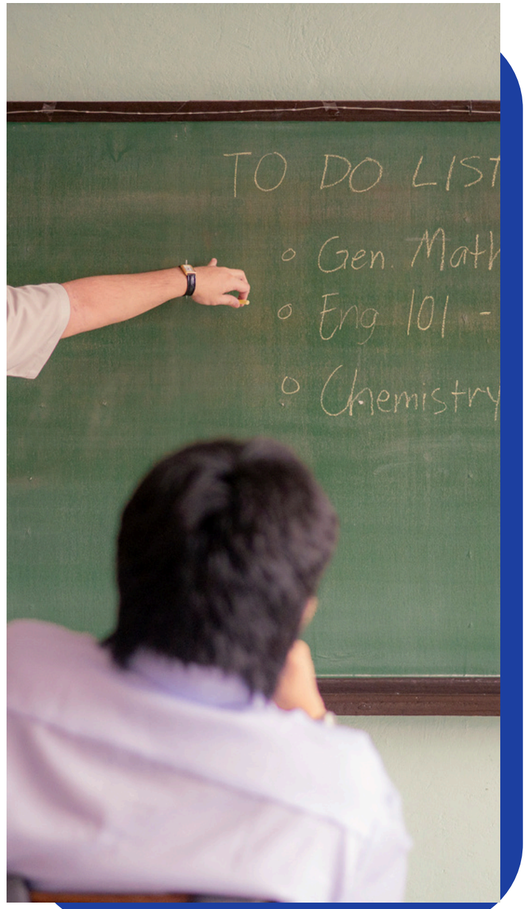


DIAGRAMME DE SÉQUENCE



Clubs & Événements

Microservice chargé de la vie associative et des activités événementielles du campus.





Gérer la vie des clubs

Un point central pour les clubs, les adhésions, les événements et les inscriptions.

CE QUE LE SERVICE PREND EN CHARGE

Des clubs actifs, des événements organisés.

Il centralise les demandes d’adhésion, les rôles des membres, les inscriptions et les listes d’attente.

CLUBS

Consulter et proposer des clubs.

ADHÉSIONS

Demander à rejoindre un club et traiter les demandes.

ÉVÉNEMENTS

Créer et consulter les événements du campus.

INSCRIPTIONS

Gérer les places disponibles et la liste d’attente.

Base de données : MySQL • Communication avec MS Réservations lorsqu’une salle est nécessaire.

Acteurs & API

Les routes REST exposent les actions principales du microservice.

LES ACTEURS

ÉTUDIANT	Consulte, propose un club, demande une adhésion et s'inscrit.
MANAGER	Gère les adhésions et organise les événements de son club.
STAFF	Crée des événements institutionnels.
ADMIN	Valide les clubs et crée des événements institutionnels.

EXEMPLES D'ENDPOINTS

CLUBS

GET	**/api/clubs**	Consulter les clubs
POST	**/api/clubs**	Proposer un club
PATCH	**/api/clubs/{id}/approve**	Valider un club

ÉVÉNEMENTS & INSCRIPTIONS

POST	**/api/events**	Créer un événement
GET	**/api/events**	Consulter les événements
POST	**/api/events/{id}/registrations**	S'inscrire
DELETE	**/api/events/{id}/registrations/me**	Annuler l'inscription

GET = consulter • POST = créer • PATCH = modifier • DELETE = supprimer/annuler

Comment le microservice fonctionne

Les diagrammes résument les données principales et le scénario de création d'un événement.

DIAGRAMME DE CLASSES

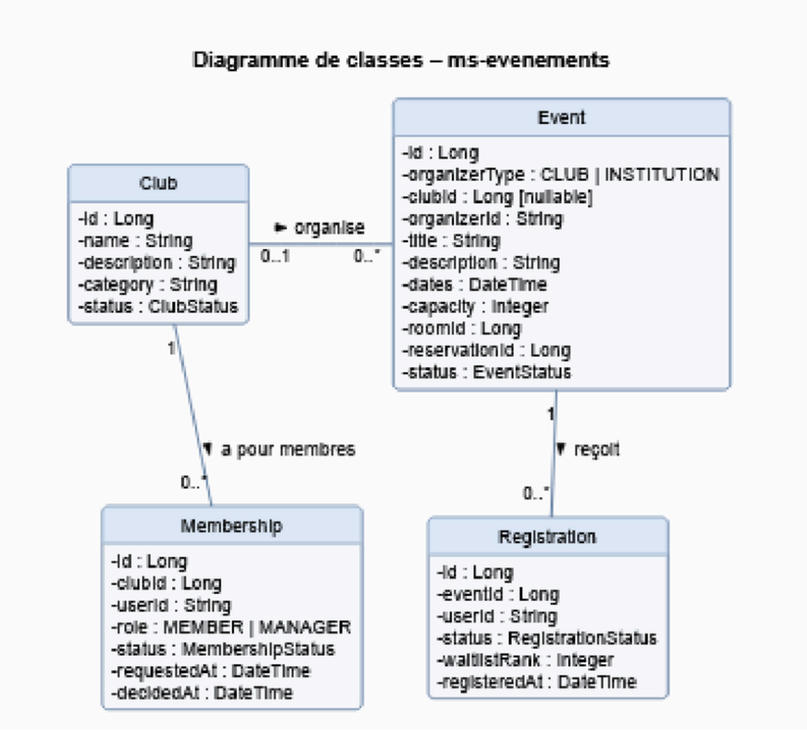
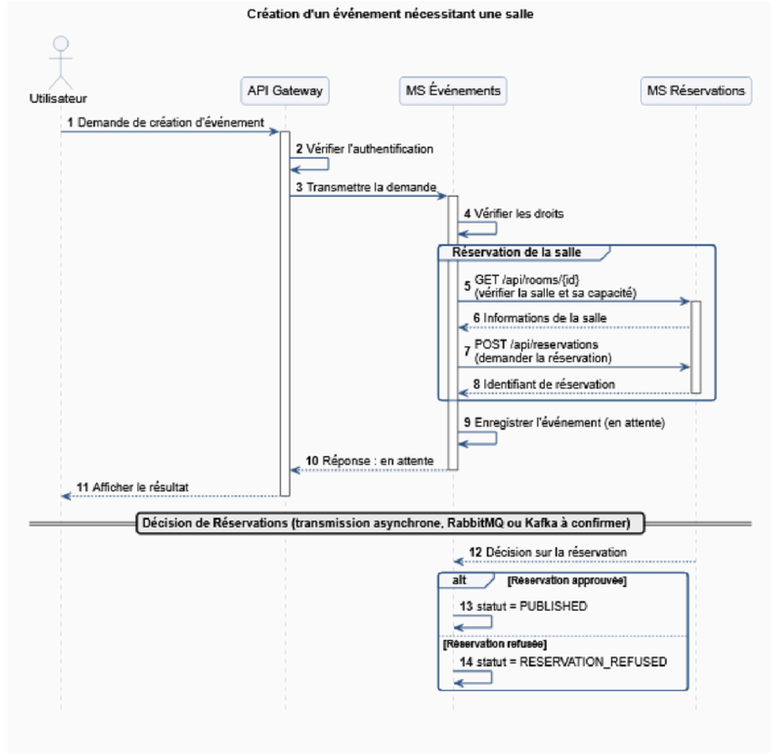


DIAGRAMME DE SÉQUENCE



Échange avec MS Réservations : vérification de la salle puis demande de réservation.

Microservice Annonces (ms-annonces)

gérer les annonces et les articles de blog de Smart Campus.

Types de contenus

ANNOUNCEMENT 01

annonces officielles, clubs et événements.



BLOG_POST 02

actualités et comptes rendus.

Fonctionnalités principales

- Création et publication
- Gestion des brouillons
- Validation par l'administration
- Planification et expiration automatiques
- Épinglage et recherche des contenus

Modèle de données et API REST et Acteurs



Les routes REST exposent les actions principales du microservice.

LES ACTEURS

ÉTUDIANT	Consulte les annonces et les articles accessibles selon son audience.
MANAGER (RESPONSABLE DE CLUB)	Crée des annonces et des articles pour son club, soumis à la validation de l'administration.
STAFF	Crée et publie directement les annonces officielles selon ses autorisations.
ADMIN	Crée les annonces officielles et valide ou refuse les contenus des clubs.

EXEMPLES D'ENDPOINTS

GESTION DES CONTENUS

POST	**/api/contents**	Créer une annonce ou un article
GET	**/api/contents**	Consulter et rechercher les contenus
GET	**/api/contents/{id}**	Consulter un contenu

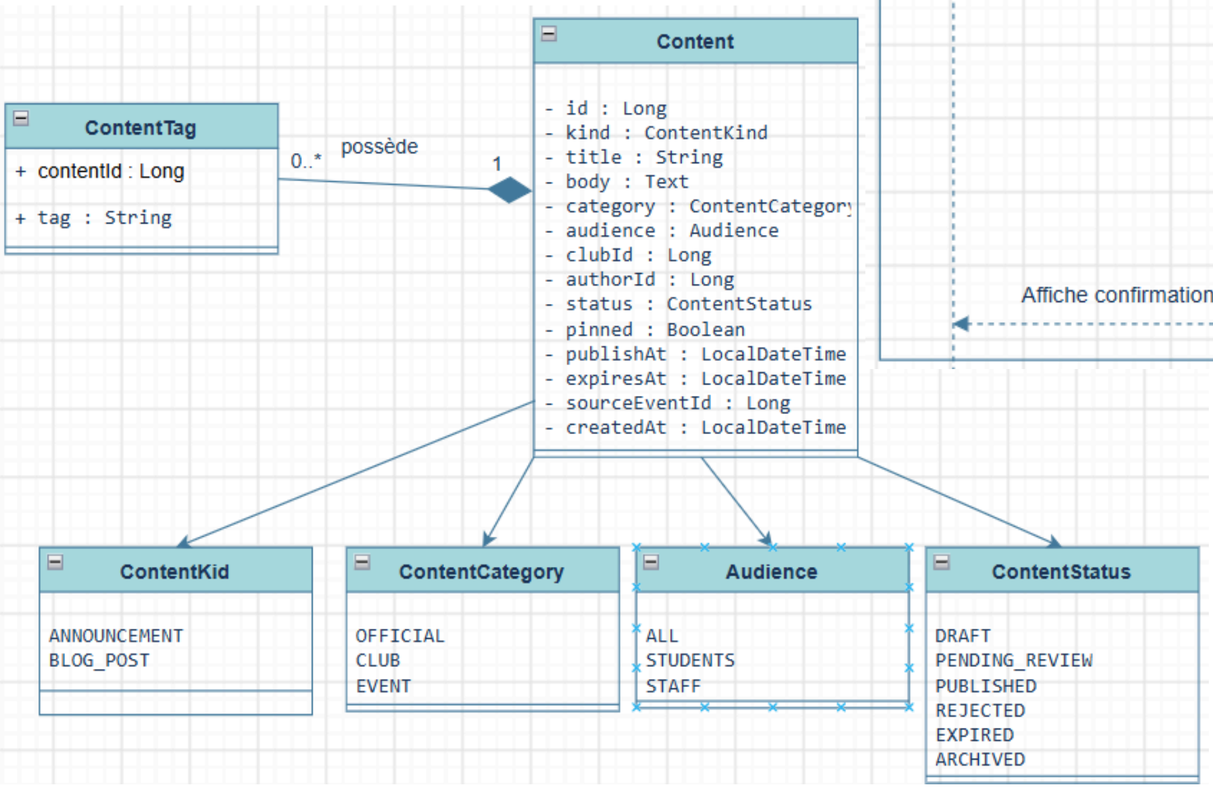
PUBLICATION & VALIDATION

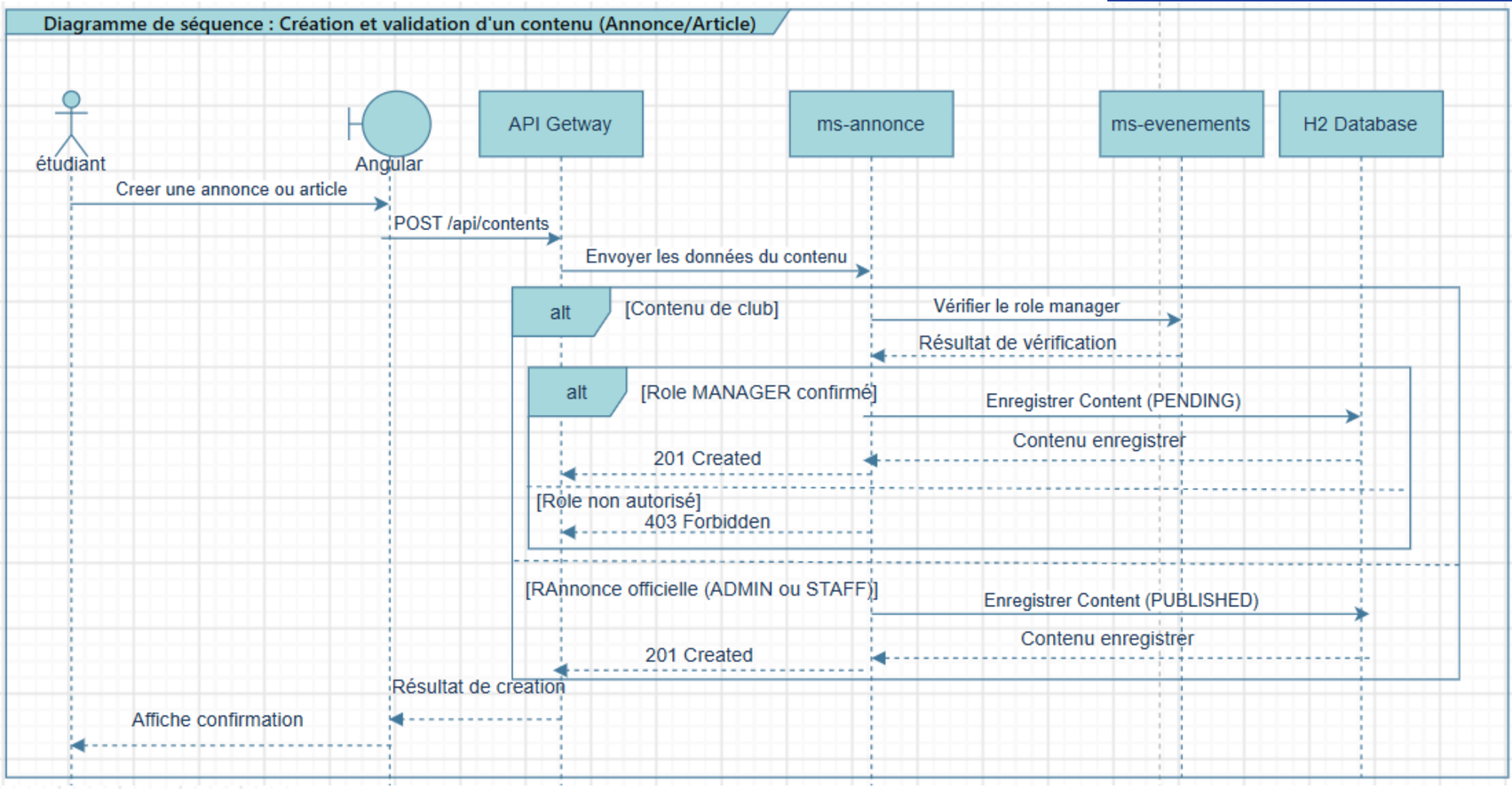
PATCH	**/api/contents/{id}/publish**	Publier un contenu
PATCH	**/api/contents/{id}/approve**	Valider un contenu
PATCH	**/api/contents/{id}/reject**	Refuser un contenu

GESTION DES ANNONCES

PATCH	**/api/contents/{id}/pin**	Épingler une annonce importante

Modèle de données et API REST et Acteurs





Communication entre les microservices et sécurité



Sécurité :

- **Keycloak : authentification**
- **STUDENT : consultation**
- **STAFF : création des annonces officielles**
- **ADMIN : validation et gestion**
- **MANAGER : rôle métier vérifié auprès de ms-evenements**

Infrastructure commune :

Spring Cloud Gateway

Eureka

Config Serve

Rabbit MQ

docker

Objets perdus

Microservice chargé de la déclaration, de la correspondance et de la restitution des objets du campus.



ms-objets-perdus

Retrouver les objets perdus

CE QUE LE SERVICE PREND EN CHARGE

Des objets déclarés, des objets rendus.

Deux types de déclarations : LOST pour un objet perdu, FOUND pour un objet trouvé, avec une question de vérification.

DÉCLARER

Signaler un objet perdu ou trouvé.

RECHERCHER

Filtrer par type, catégorie, statut, mots-clés ou date

RAPPROCHER

Proposer une correspondance entre un objet perdu et un objet trouvé.

RESTITUER

Vérifier la propriété, puis remettre l'objet au bureau de l'administration.

Sans réclamation après 30 jours, l'objet passe à EXPIRED (planificateur @Scheduled).

Du signalement à la restitution

1

OPEN

L'objet est déclaré et attend une correspondance.

→

2

MATCH_PROPOSED

Une correspondance est proposée au propriétaire.

→

3

CLAIM_PENDING

Une réclamation répond à la question de vérification.

→

4

RETURNED

La remise est enregistrée au bureau de l'administration.

Base de données : MongoDB • Messages asynchrones vers MS Notifications

Acteurs & API

Les routes REST exposent les actions principales du microservice.

LES ACTEURS

ÉTUDIANT (STUDENT)

Déclare, consulte et réclame un objet trouvé.

PERSONNEL (STAFF)

Déclare et consulte, comme un étudiant.

ADMINISTRATEUR (ADMIN)

Tranche les réclamations et enregistre la restitution.

EXEMPLES D'ENDPOINTS

POST	/api/items	Déclarer un objet
GET	/api/items	Rechercher des objetst
GET	/api/items/{id}	Consulter un objet
GET	/api/items/{id}/matches	Consulter les correspondances
POST	/api/items/{id}/claims	Déposer une réclamation
PATCH	/api/claims/{id}/approve	Approuver une réclamation
PATCH	/api/claims/{id}/reject	Refuser une réclamation
PATCH	/api/items/{id}/return	Enregistrer la restitution

GET = consulter • POST = créer • PATCH = modifier(partiel) • DELETE = supprimer

Comment le microservice fonctionne

Les diagrammes résument les données principales et le scénario de declaration d'un objet.

DIAGRAMME DE CLASSES

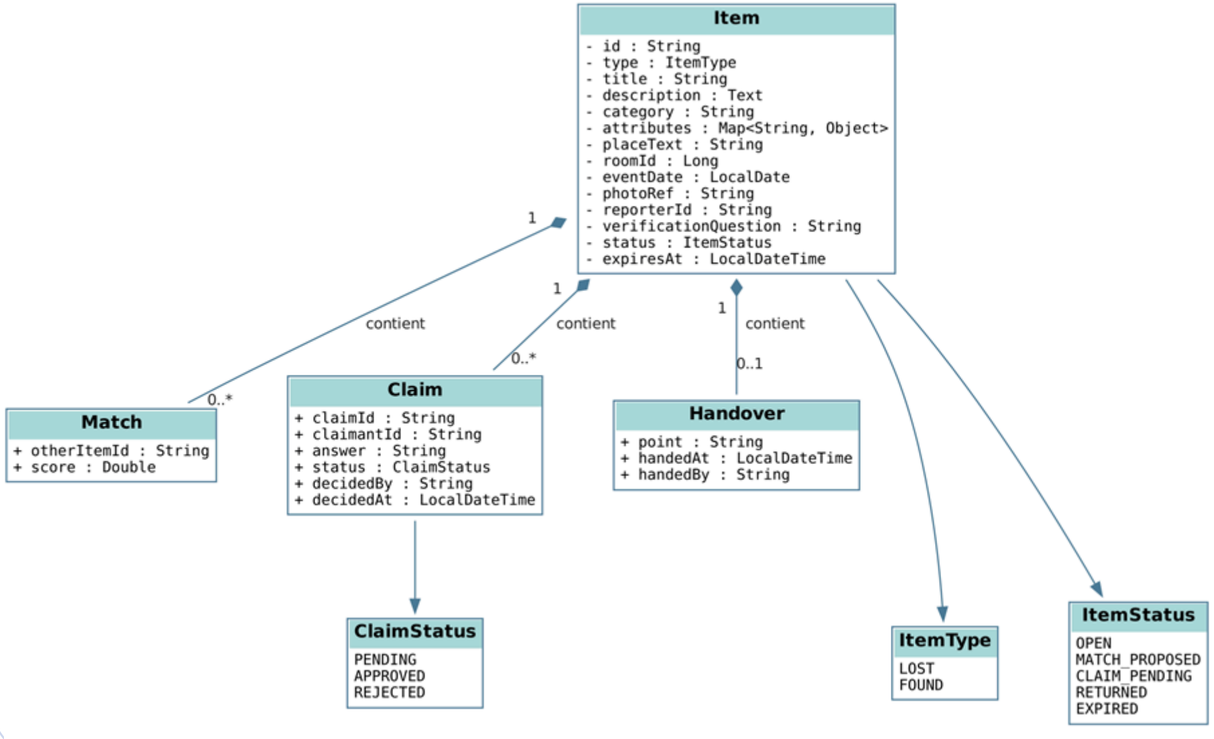
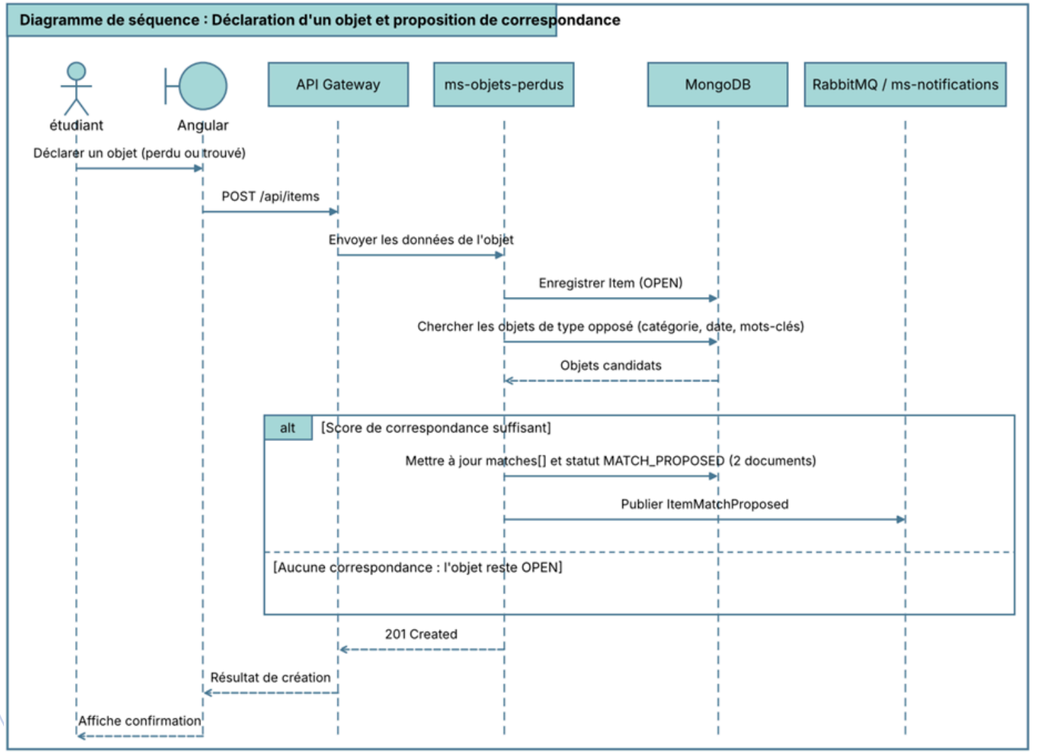


DIAGRAMME DE SÉQUENCE



Microservice notifications

Un service commun pour centraliser les notifications et les rappels de Smart Campus.

CE QUE LE SERVICE PREND EN CHARGE

Des utilisateurs informés, au bon moment.

Le service reçoit les événements des autres microservices, crée les notifications et les transmet aux utilisateurs concernés.

NOTIFICATIONS

Créer et conserver les notifications de chaque utilisateur.

PRÉFÉRENCES

Permettre à l'utilisateur d'activer ou de désactiver certaines catégories.

TEMPS RÉEL

Envoyer les nouvelles notifications avec Server-Sent Events.

RAPPELS

Programmer un rappel avant le début d'un événement.

Base de données : H2

Acteurs et API

Les utilisateurs consultent leurs notifications, tandis que les microservices transmettent les événements métier.

LES ACTEURS

UTILISATEUR Consulte ses notifications et les marque comme lues.

STAFF Consulte les ressources et crée une demande de réservation.

SYSTÈME DE RAPPEL Crée automatiquement une notification avant le début d'un événement.

EXEMPLES D'ENDPOINTS

NOTIFICATIONS

GET	/api/notifications	Lister les notifications
GET	/api/notifications/{id}	Consulter une notification

PRÉFÉRENCES ET TEMPS RÉEL

GET	/api/notification-preferences	Consulter ses préférences
PUT	/api/notification-preferences	Modifier ses préférences

GET = consulter • PUT = modifier(remplacer) • DELETE = supprimer/annuler

Comment le microservice fonctionne

Les diagrammes résument les données principales et le scénario de reception d’un évenement .

DIAGRAMME DE CLASSES

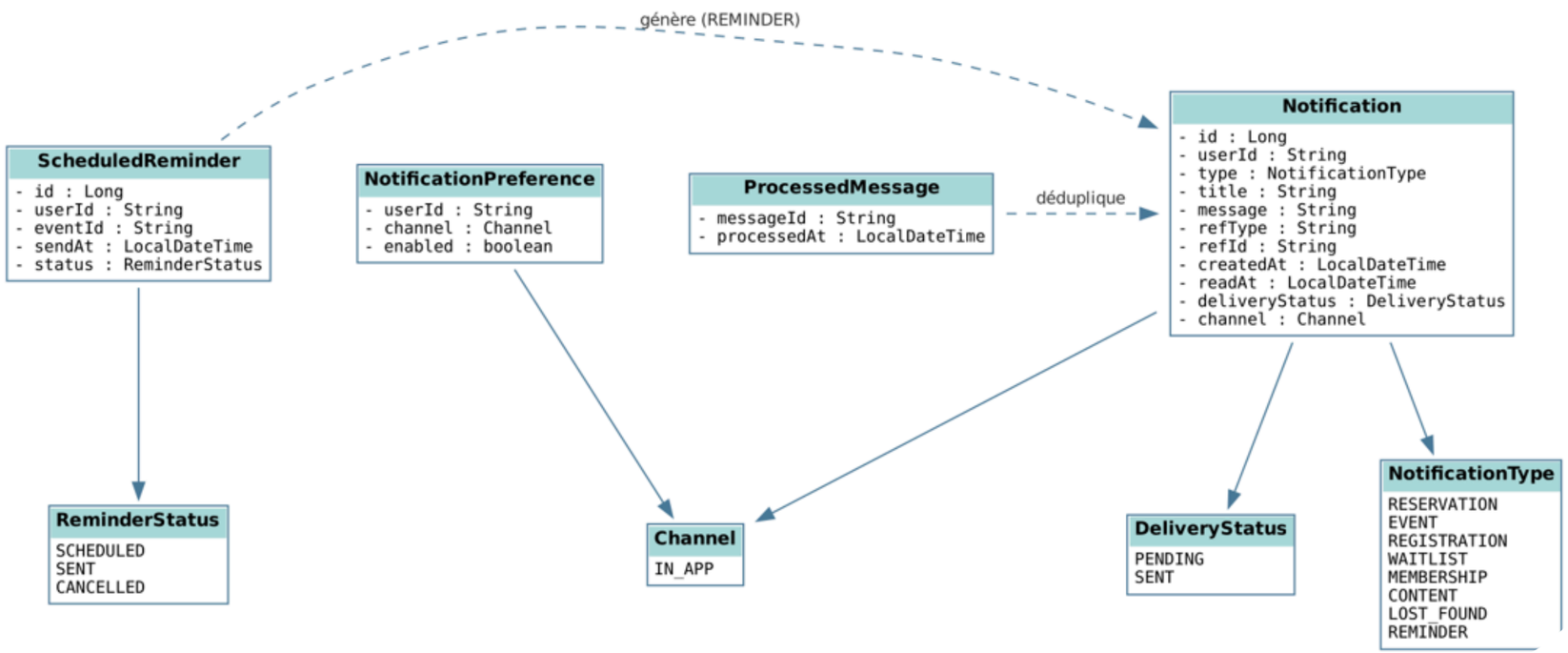
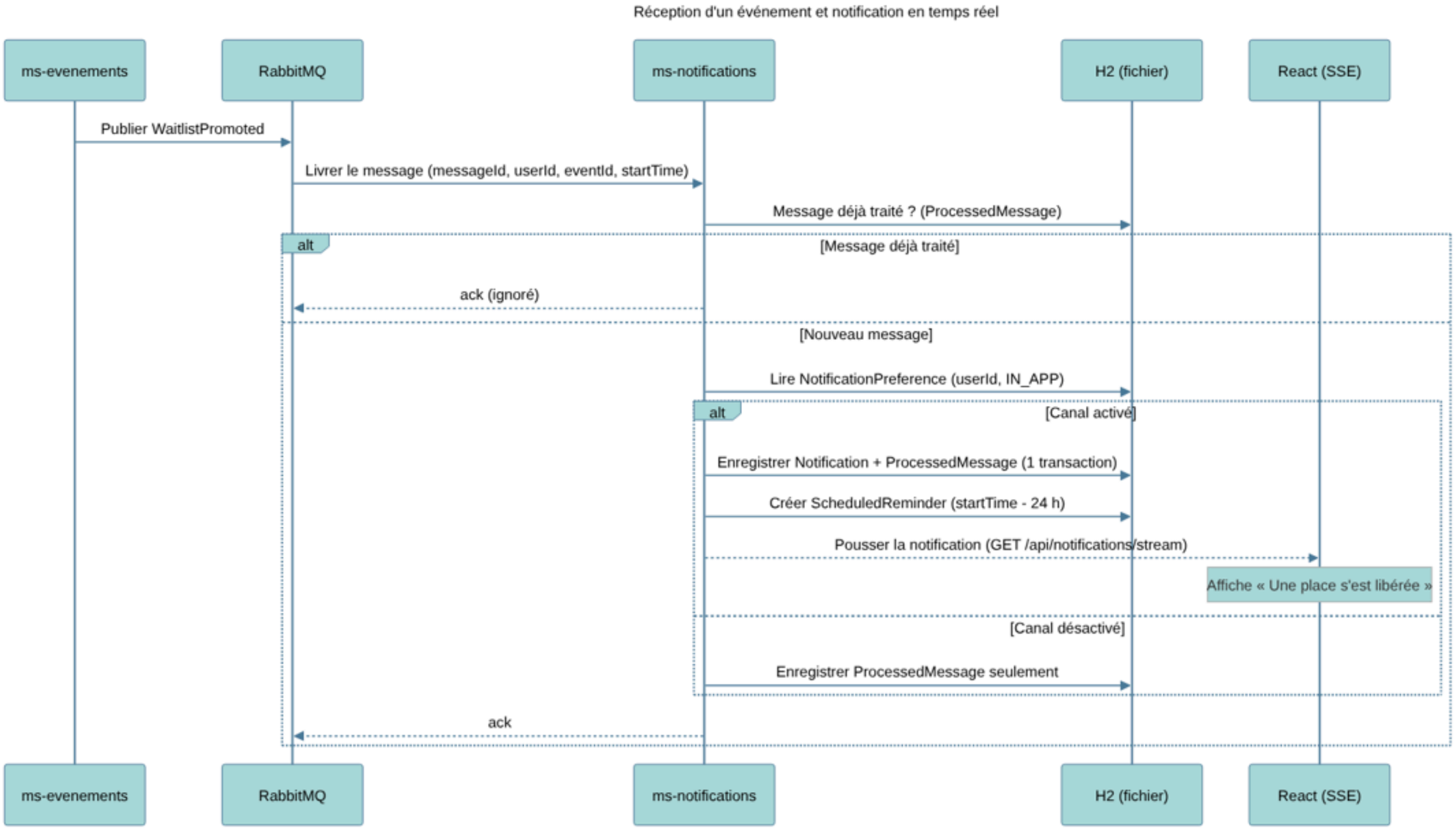


Diagramme de sequence de ms-notifications



Répartition du travail

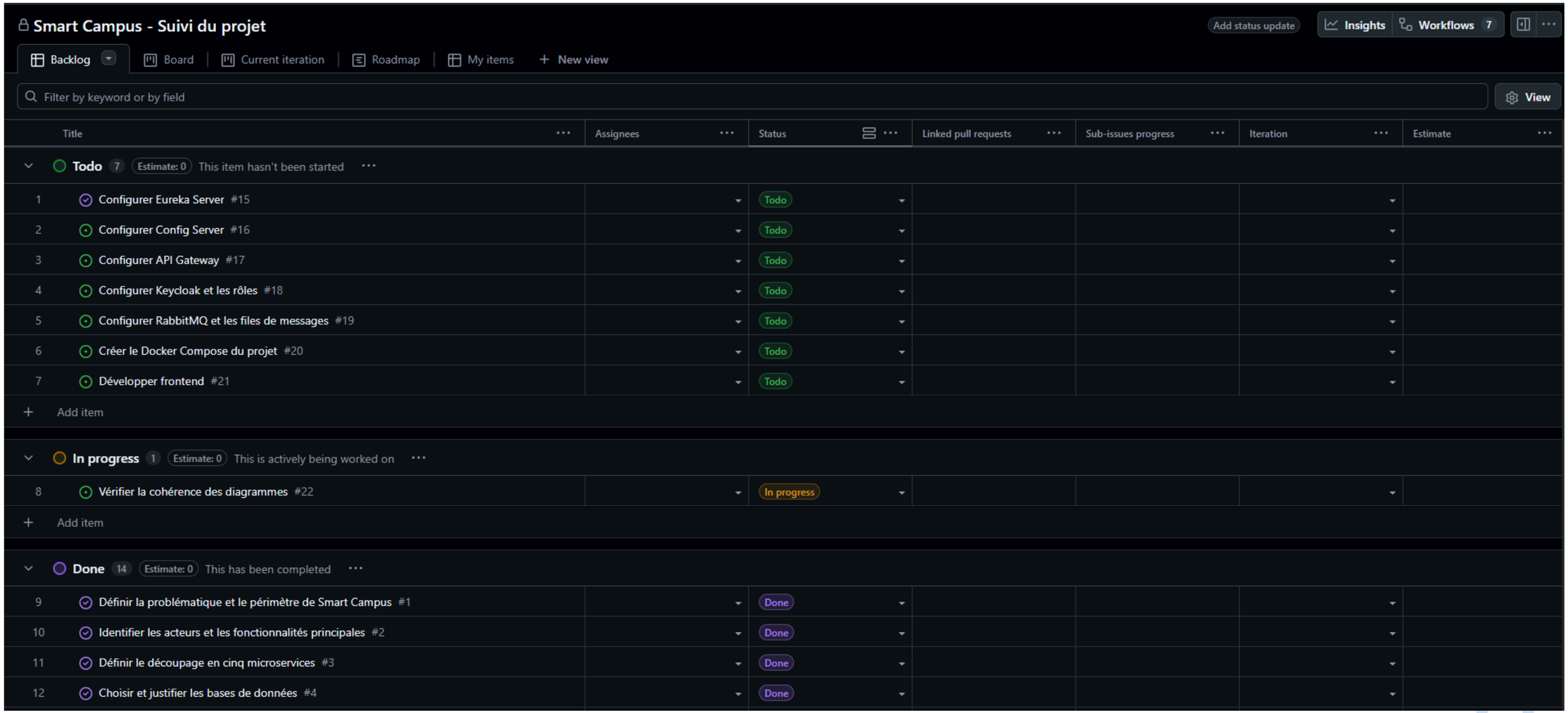
Membre	Responsabilité principale
Bechir	Réservations et PostgreSQL
Dhia	Clubs, événements et MySQL
Yassine	Annonces et H2
Chadha	Objets perdus et MongoDB
Toute l’équipe	Notifications et infrastructure commune

Tâches communes :

L’architecture globale, l’API Gateway, Eureka, Config Server, Keycloak, RabbitMQ, Docker Compose, la documentation et la présentation sont réalisés collectivement.

Le frontend repose sur une seule application Angular. Chaque membre développe le module correspondant à son domaine.

Organisation et suivi du projet



Répartition des tâches et suivi de l’avancement avec GitHub Projects



CONCLUSION GÉNÉRALE

Smart Campus propose une plateforme unique pour simplifier et moderniser la vie universitaire.

Une solution qui permet de :

01 Centraliser les services et les informations du campus

02 Faciliter la réservation et la gestion des événements

03 Renforcer la communication entre les acteurs

04 Offrir une architecture modulaire et évolutive

Merci pour votre attention

Smart Campus

Plateforme numérique de gestion de la vie universitaire